CSE 3101 Compiler Design Lecture Notes
Introduction to Compilers

A compiler is a program that translates source code written in a high-level language into an equivalent program in a lower-level target language, usually machine code. Unlike an interpreter, which executes the source statement by statement at run time, a compiler performs its translation ahead of execution and reports errors before the program runs.

The compilation process is commonly divided into an analysis part, known as the front end, and a synthesis part, known as the back end. The front end understands the source program and builds an intermediate representation. The back end generates target code from that representation.

A symbol table is maintained throughout compilation. It records identifiers together with attributes such as type, scope, and storage location.

CSE 3101 Compiler Design Lecture Notes
Lexical Analysis

Lexical analysis is the first phase of a compiler. The lexical analyzer, also called a scanner, reads the stream of source characters and groups them into meaningful sequences called lexemes. For each lexeme it produces a token that the parser consumes.
A token has a name and an optional attribute value. Common token classes are keywords, identifiers, literals, operators, and punctuation. White space and comments are normally discarded at this stage.
Lexical patterns are described with regular expressions. Tools such as Lex and Flex generate a scanner automatically from a set of regular expression rules.

A regular expression can be converted into a nondeterministic finite automaton using Thompson's construction. A nondeterministic finite automaton may have several possible transitions on one input symbol, and may also have epsilon transitions that consume no input.
The subset construction algorithm converts a nondeterministic automaton into an equivalent deterministic finite automaton, in which every state has exactly one transition for each input symbol. A deterministic automaton is faster to simulate, although it can require exponentially more states.
The number of states of a deterministic automaton can be reduced by state minimization, which merges states that cannot be distinguished by any input string.

CSE 3101 Compiler Design Lecture Notes
Context-Free Grammars and Syntax Analysis

Syntax analysis, or parsing, is the second phase of compilation. The parser takes the token stream and verifies that it conforms to the grammar of the language, producing a parse tree.

Programming language syntax is specified with a context-free grammar, which consists of terminals, nonterminals, a start symbol, and production rules. A grammar is ambiguous if some string has more than one leftmost derivation, and therefore more than one parse tree. The dangling else problem is a classic example of ambiguity.

Left recursion occurs when a nonterminal can derive a string beginning with itself. It must be eliminated before a top-down parser can be used, because such a parser would otherwise loop forever.

CSE 3101 Compiler Design Lecture Notes
Top-Down Parsing

A top-down parser builds the parse tree from the root toward the leaves, simulating a leftmost derivation. Recursive descent parsing implements each nonterminal as a procedure and may require backtracking.
A predictive parser is a recursive descent parser that needs no backtracking because it chooses a production by looking at the next input token. An LL(1) grammar is one that can be parsed this way: scanning the input from left to right, producing a leftmost derivation, using one token of lookahead.
Construction of an LL(1) parsing table relies on the FIRST and FOLLOW sets. FIRST of a string is the set of terminals that can begin strings derived from it. FOLLOW of a nonterminal is the set of terminals that can appear immediately to its right in some sentential form.

A bottom-up parser begins at the leaves and works toward the root, reducing the input string to the start symbol. It traces a rightmost derivation in reverse.
Shift-reduce parsing uses a stack and two main actions. A shift pushes the next input token on the stack. A reduce replaces a handle on top of the stack with the nonterminal on the left side of a production.
LR parsers are the most general efficient shift-reduce parsers. SLR, canonical LR, and LALR differ in how they build the parsing table. LALR is used by the tool Yacc because its tables are much smaller than canonical LR tables. A shift-reduce conflict arises when the parser cannot decide whether to shift or to reduce.

CSE 3101 Compiler Design Lecture Notes
Semantic Analysis and Intermediate Code

Semantic analysis checks that a syntactically correct program is meaningful. Its most important task is type checking, which ensures that operators are applied to compatible operands. Syntax-directed translation attaches semantic rules to grammar productions; synthesized attributes are computed from a node's children, and inherited attributes come from its parent or siblings.

Compilers usually generate an intermediate representation before target code. Three-address code is a popular form in which each instruction has at most three operands, such as x = y + z. Common implementations are quadruples and triples. An intermediate form makes the compiler easier to retarget to new machines.

Code optimization improves intermediate code without changing the meaning of the program. Constant folding evaluates constant expressions at compile time. Dead code elimination removes statements whose results are never used. Common subexpression elimination avoids recomputing a value that is already available. Loop invariant code motion moves computations that do not change out of a loop.

The code generator maps intermediate code to target instructions. Register allocation decides which values stay in the limited set of machine registers and is often modelled as graph coloring: an interference graph is built, and registers are assigned as colors so that interfering values differ.